System Hardware

SOEN228

Lab Report #0

Student Name	Student ID	Lab section	Lecture section
Michaël Gugliandolo	40213419	C-Z (lab D)	U
Jean-Nicolas Sabatini-Ouellet	40207926	B	U

Department of Computer Science & Software Engineering

Concordia University

Winter 2022

Objectives **2**

Introduction **2**

Experimental Results and Questions **3**

Experiment 0 3

Part 1 3

Part 2 4

Experiment 1 5

Part 1 5

Part 2 6

Part 3 7

Discussion **7**

Experiment 0 7

Part 1 7

Part 2 8

Experiment 1 8

Part 1 8

Part 2 8

Part 3 8

Conclusions **8**

1. Objectives

This lab consists of two lab experiments, where the main objective is to get familiar with the Atanua software by constructing basic components, such as a half adder, SR latches and flip-flops. These components utilize different types of logic gates, such as AND, OR and NOT gates. Furthermore, basic components like these are the building blocks for more complex parts, and it is essential to get used to building them.

2. Introduction

In this lab, chips 7404 (NOT), 7408 (AND) and 7432 (OR) were needed. These chips have pins. Starting from the top left, the 7408 and 7432 chips have two input pins and one output pin, and this layout is repeated for the other pins in the chips. On the other hand, the 7404 has a repeating pattern of 1 input and 1 output.

The 7404 chip's (NOT gate) output is the opposite of the input, which means that an input of 1 will have an output of 0 and vice-versa. The 7408 (AND gate) chip needs its two inputs to be 1 for its output to be 1. Otherwise, its output will be 0. Thirdly, the 7432 chip (OR gate) needs at least one input to equal 1 for its output to be 1.

Other useful logic gates are XOR, NAND and NOR gates. An XOR gate will output 1 if its two inputs are different. NAND and NOR gates are AND and OR gates with their outputs inverted.

By combining these chips or gates, we can build more complex circuits.

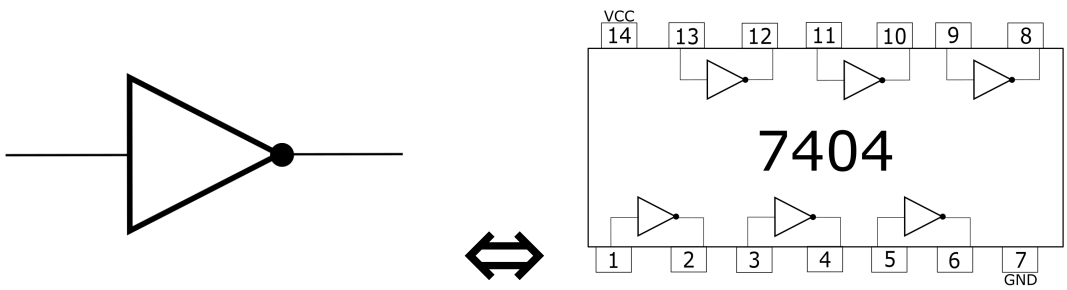
Furthermore, in the flip-flop experiment, we used a clock. A clock is a type of input that alternates its value every set amount of time (ie alternates between 0 and 1 each millisecond). It is used in synchronous circuits so that inputs in the memory circuits, when changed, are all synchronized together.

3. Experimental Results and Questions

Experiment 0

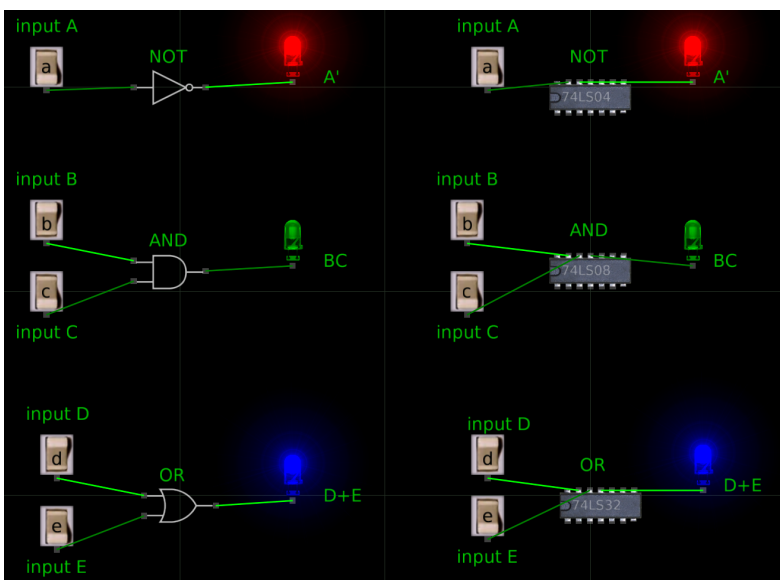
Part 1

We first have to use the basic logic gates to copy the 7404 (NOT), 7408 (AND) and 7432 (OR) chips. Each chip contains multiple logic gates inside, but we don't need all of the pins. For example, for the NOT gate and chip:



<u>1. Using Atanua, build the same circuit using basic menu and compare between two models.</u>

<u>2. Using extremal pin under MISC menu draw the three logic gates circuit with best documentation using Label under Misc menu.</u>



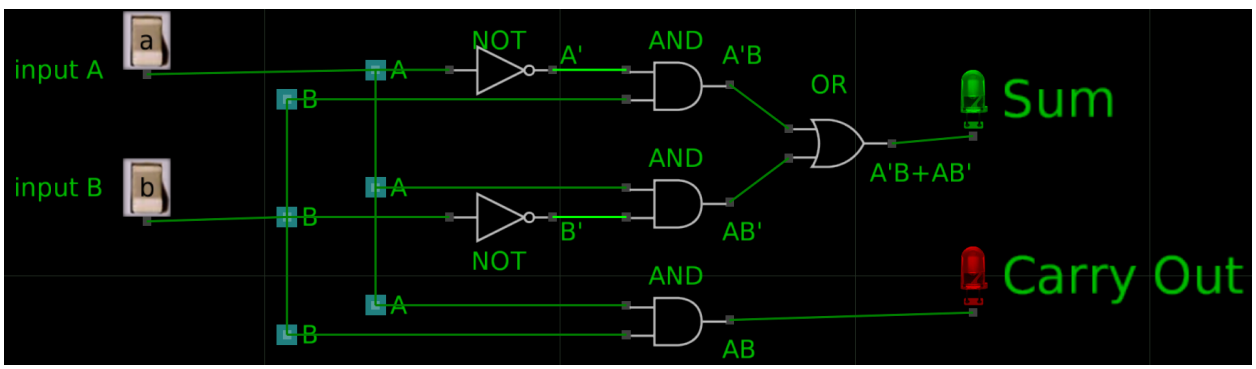
The chips behave exactly the same way as the logic gates, but they're more useful practically. For example, you can compute many AND gates in one single 7408 chip, which saves space and money. However, it's a lot easier to understand circuits built with logic gates, so it's better to use basic gates to represent complicated circuits, then convert them into chips for compactness.

Part 2

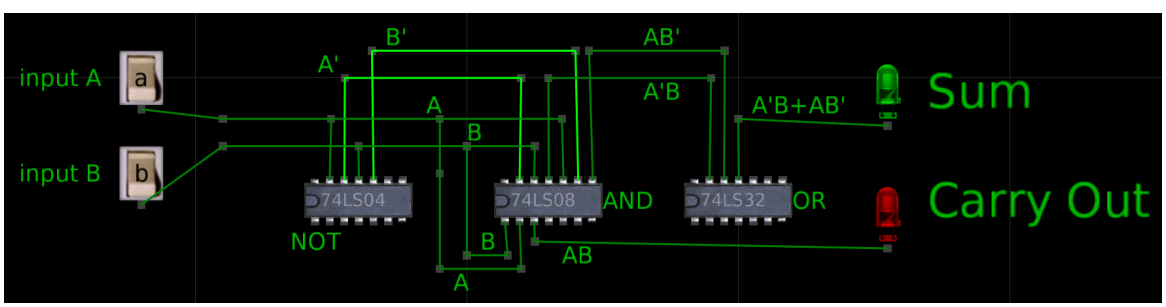
Secondly, we have to build a half adder. In its simplest form, it takes two 1-bit numbers as inputs and outputs their sum and if there's a carry out. It requires an XOR gate, built with 2 NOT, 2 AND and 1 OR gate, and another AND gate. The XOR gate (true if both inputs are different) computes the sum and the AND gate (true if both inputs are true) computes the carry out, as indicated by the following truth table:

A	**B**	**Sum**	**Carry out**
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

We can use minterms to transform the XOR into simple logic gates. The sum is 1 when A'B or AB', so we can say that A⊕B = A'B+AB'. Here's the circuit in logic gates:



Each type of logic gate is in its own column to easily see how the chips will be connected. Here's the same circuit with chips instead of logic gates:



<u>1. What is the maximum input bits half adder can use; can it support adding 3 bits?</u>

The maximum input bits a half adder can take is 2 bits (1 for each input). Since a half adder can only add up to 10 in binary or 2 in decimal format, a third bit would not fit into the circuit.

Precisely, to add to 11 in binary, both the sum and carry out would have to equal 1. However, the carry out is 1 only when A and B are true, which means that the sum will be 0 by definition since it uses an XOR gate (true when both inputs are different).

2. Can a half adder be used to as a building block for a larger combinational circuit? Explain.

Yes, a half-adder is the building block of a full adder. A full adder is built on top of a half adder, and it can be used to add up to 11 in binary or 3 in decimals. They can also be linked together to add larger numbers, and computers will use circuits like these to perform additions and other operations.

Experiment 1

Part 1

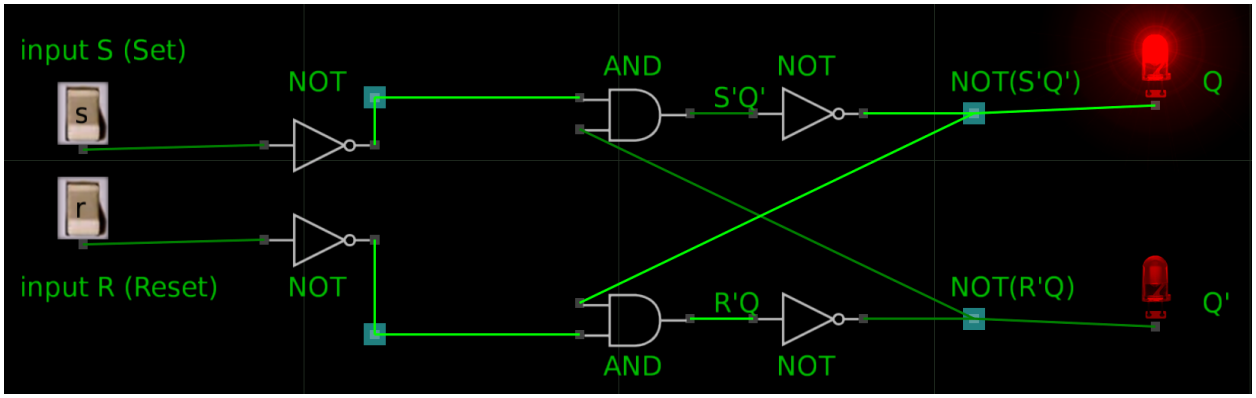
In the second lab experiment, we have to build latches and flip-flops. These components differ from half adders because they can keep values, while half adders only compute inputs into outputs. Also, latches modify their stored bit immediately when inputs are given, while flip-flops are edge-triggered, meaning that they are synchronized with a clock and only change their bit when the clock changes signal.

In the first place, we have to build an S-R latch with NAND gates. There are 2 inputs: S (set) and R (reset), and 2 outputs: Q (stored bit) and Q' (inversed bit). Inputting 1 for both inputs results in an invalid state. Otherwise, if S=1, the stored bit (Q) changes to 1, and if R=1, Q changes to 0. It's worth noting that in a NAND S-R latch, the inputs are inverted.

S'	**R'**	**Q**	**Q'**
1	1	No change	
1	0	0	1
0	1	1	0
0	0	Invalid state	

1. What is the difference between the NAND and NOR implementations of a S-R latch?

The NAND and NOR S-R latches don't have the same exact circuit, since the NAND version uses 2 NAND gates, while the NOR uses 2 NOR gates. Furthermore, the 2 versions have the same outputs if the S and R input bits are the same. However, for the NAND latch, the basic inputs have to be inverted and as such, the outputs are also inverted. As shown below, we had to invert the inputs before passing them into the S-R latch.



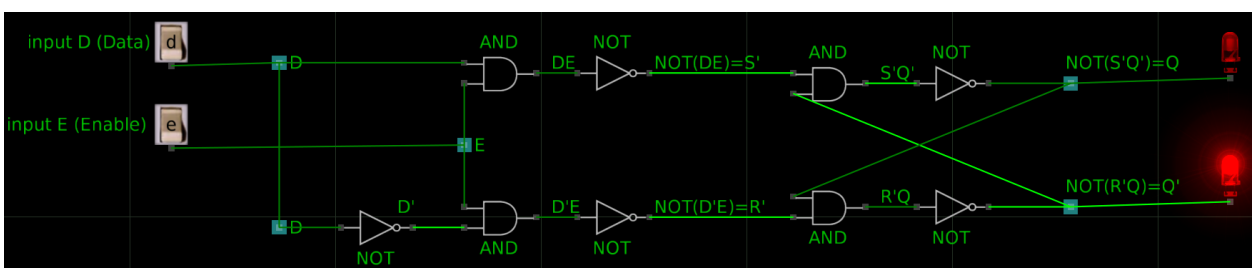
Part 2

On another note, a D latch is a latch that's easier to control, since it doesn't have an invalid state. Q takes the data bit's (D) value if the other input bit equals 1, called E (enable). If E=0, then Q doesn't change. In other words, E controls if D can change the output.

D	**E**	**Q**	**Q'**
X	0	No change	
0	1	0	1
1	1	1	0

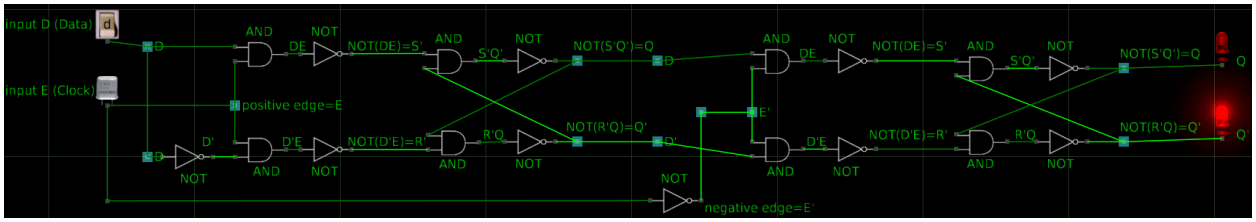
(X means that this value doesn't affect the output)

As shown below, a D Latch is built on top of an S-R NAND Latch.

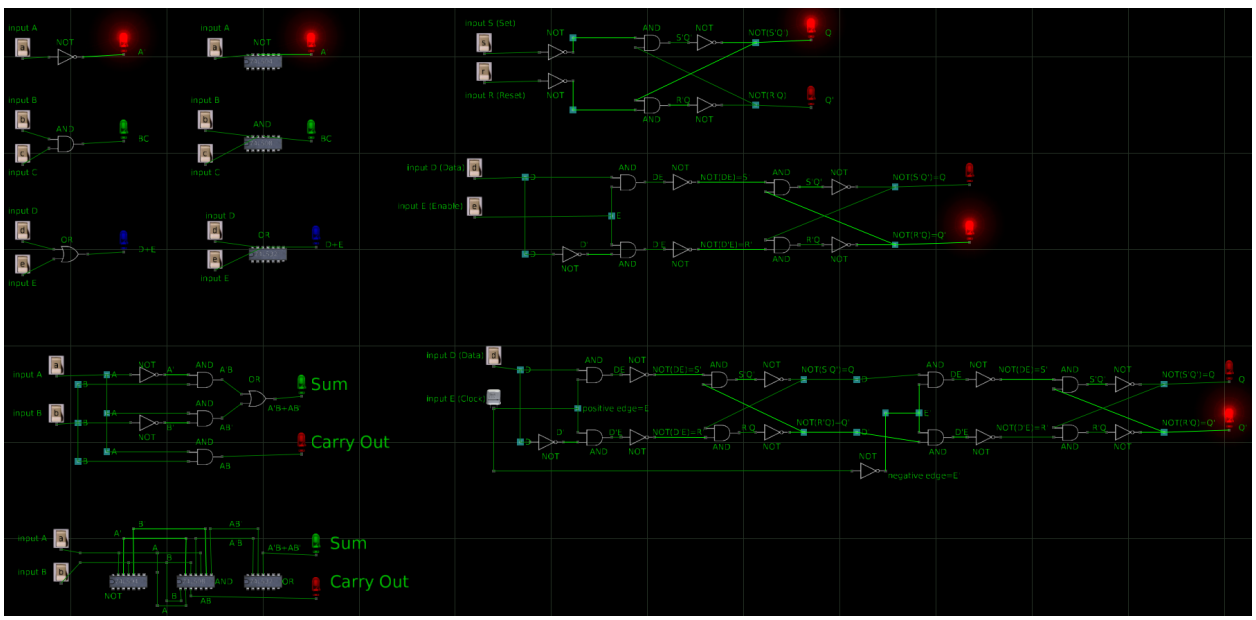


Part 3

To make a more stable memory unit, we can synchronize them with the change of a clock's signal from high to low or from low to high, called positive-edge or negative-edge triggered respectively. By replacing E in the D latch with the clock, we have created flip-flops. We have to build a negative-edge flip-flop by linking two D latches. The first one will be positive-edge and the second one, negative-edge; the Q of the first latch will be the D of the second one.



4. Discussion



Experiment 0

Part 1

It's more compact to use chips rather than logic gates. The wires could've also been positioned closer to the chips to further demonstrate it. By saving space, we could also save money, which is why computers are always built with chips. It is important to note however that the use of logic gates for demonstrations should be preferred, as it is easier to navigate the circuit.

Part 2

The half adder made of chips is rather messy, and it can be difficult to understand at first what's going on. Since chips have multiple pins and thus, multiple gates, reusing a chip multiple times makes the overall circuit very messy and hard to comprehend. This shows the importance of conceptualizing the circuit with logic gates to make it easy to understand and to change the gates into chips.

Experiment 1

Part 1

The basic version of a NAND S-R latch doesn't have anything to protect it against the inputs leading to an invalid state, which makes it vulnerable. Also, a NOR S-R latch doesn't require the inputs to be inverted, which could be an advantage, since it requires fewer gates. Furthermore, by having invalid states, the S-R latch has more disadvantages when used fundamentally. However, this latch is the building block of a much better latch, the D-Latch.

Part 2

We can see that a D Latch is easier to control compared to an S-R latch, as the input E controls if D can change the output. Moreover, a D Latch does not have an invalid state, which makes it a better choice to use in most circuits compared to the S-R latch. Unfortunately, it requires more logic gates.

Part 3

In the flip-flop, the clock is used to regulate the rate of change of the stored bit (Q), and when used in multiple circuits, it can synchronize all of the outputs. In retrospect, a flip-flop has a very similar function as a D-Latch, the only difference being that instead of an input to decide when the output changes, a clock is used.

5. Conclusions

In the first lab, we practiced using the Atanua software by comparing common chips with their equivalent logic gates. We also designed a half adder with logic gates and built it using 3 chips. It is clear that half adders like ours are the base for more complex circuits, even though they can only add two bits in their most basic forms.

In the second lab, we built sequential circuits. Starting with an S-R latch, we realized that its invalid state isn't ideal, so we built a D latch. D latches are easier to control, among other things

because of their enable input (E) and because they have no invalid state. This can be improved upon by replacing E with a clock to make an edge-triggered flip-flop. Clocks synchronize outputs, and large circuits are much stabler with them.